

Application Program Development

Input, Processing and Output

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2026



EAST TEXAS A&M

Designing a Program (1 of 2)

Program Development Cycle

① Design the Program

- Design is the most important part of program development
- Understand the task that the program is to perform

② Write the Code

- Use a programming language, like Python

③ Correct Syntax Errors

④ Test the Program

- A *logic error* is a mistake that does not prevent program from running, but causes incorrect results.
- Unit testing, module testing, system testing, stress testing

⑤ Correct Logic Errors

- *debug* find and correct logic errors



Designing a Program (2 of 2)

- Design is the most important part of program development
- Understand task that the program is to perform
 - Work with customers and users
 - Create software requirements
- Determine the steps that must be taken to solve the problem
 - Break down required task into series of steps
 - Create an algorithm, listing logical steps that must be taken
- **Algorithm:** set of well-defined logical steps that must be taken to perform a task.



Pseudocode and Flowcharts

Pseudocode: fake code, informal, no syntax rules

Input the hours worked

Input the hourly pay rate

Calculate gross pay as hours worked multiplied by pay rate

Display the gross pay

Flowchart: diagram graphically depicting steps in a program

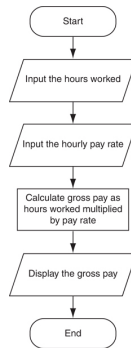


Figure 1: Flowchart for a pay calculating program (Gaddis, 2021, pg.57)



Input, Processing and Output

- Typically computer performs three-step process
 - ① Receive input
 - Input: any data that the program receives while it is running
 - ② Perform some process on the input
 - Example: mathematical calculation
 - ③ Produce output

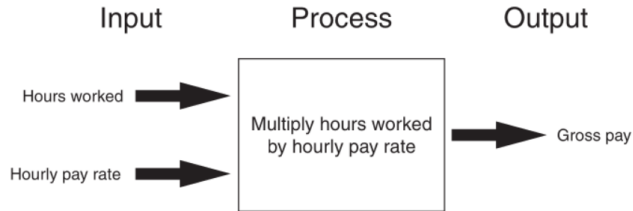


Figure 2: The input, processing, and output of the pay calculating program (Gaddis, 2021, pg. 68)



Displaying Output with the print() Function

- **Function:** piece of prewritten **reusable** code that performs an operation
- `print()` function: displays output on standard out (terminal)
- **Argument:** data given as input to a function
 - Example: give a string to be printed on terminal
- Statements in a program execute in the order that they appear (sequential execution)

```
c:\Users\etamustudent> python3
>>> print('Hello world')
Hello world
>>>
```



Strings and String Literals

- **String:** sequence of characters that is used as data
- **String literal:** string that appears in actual code of a program
 - Must be enclosed in single (') or double (") quote
 - String literal can be inclosed in triple quotes (" " ") multiple lines

```
>>> print('Kate Austen')
Kate Austen
>>> print("123 Full Circle Drive")
123 Full Circle Drive
>>> contact = """
... Kate Austen
... 123 Full Circle Drive
... Asheville, NC 28899
... """
>>> print(contact)
```

```
Kate Austen
123 Full Circle Drive
Asheville, NC 28899
```



- **Comments:** notes of (human readable, English sentences) explanation within a program
 - Ignored by Python interpreter
 - Intended to communicate information to code readers and maintainers
 - Begin with a # character
- **End-line comment:** appears at the end of a line of code
 - Typically explains the purpose of that line
 - Considered bad style in many cases, do not use



Variables

- **Variable:** name that represents a value stored in the computer memory
 - Used to access and manipulate data stored in memory
 - A variable references the value it currently holds
- **Assignment statement:** used to create a variable and make it reference data
 - General format `variable = expression`
- In assignment statement, variable receiving value must be on left side
- A variable can be passed as an argument to a function
 - Variable name should not be enclosed in quote marks
- You can only use a variable if a value is assigned to it.



Variable Naming Rules

Rules for naming variables in Python:

- Variable name cannot be a Python key word
- Variable name cannot contain spaces
- First character must be a letter or underscore (not a number)
- After first character, may use letters, digits, or underscores
- Variable names are case sensitive

Variable name should reflect its use [Meaningful Variable Names](#)



Displaying Multiple Items with the print() Function

Python allows one to display multiple items with a single call to print()

- Items are separated by commas when passed as arguments
- Arguments displayed in the order they are passed to the function
- Items are automatically separated by a space when displayed on screen

```
>>> room = 503
>>> print('I am staying in room number', room)
I am staying in room number 503
```



Variable Reassignment

- Variables can reference different values while program is running
- **Garbage collection:** removal of values that are no longer referenced by variables
 - Carried out by Python interpreter
- A variable can refer to item of any type
 - Variable that has been assigned to one type can be reassigned to another type



Numeric Data Types, Literals and the str Data Type

- **Data types:** categorize value in memory
 - e.g. `int` for integer `float` for real number, `str` used for storing strings in memory
- **Numeric literal:** number written in a program
 - No decimal point considered `int`, otherwise considered `float`
- Some operations behave differently depending on data type



References

Gaddis, T. (2021). *Starting out with python* (5th). Pearson.



EAST TEXAS A&M